

SERVICIO NACIONAL DE APRENDIZAJE SENA

Centro de Gestión de Mercados, Logística y TIC



APP TRUEQUE MATCH

Desarrollo de los Módulos Móviles a partir de los Requerimientos del Proyecto

Evidencia GA8-220501096-AA2-EV02

Estudiante: Gerson Jonnathan López Oviedo

Ficha: 3186647

Instructor: Arnaldo Alfonso Montiel Brun

Programa: Tecnólogo en Análisis y Desarrollo de Software

Bogotá, Colombia. 2026

Contenido

Contenido.....	2
1. Introducción	4
2. Qué me pide esta evidencia, explicado en mis palabras	5
3. Metodología de desarrollo: Scrum aplicado al módulo móvil	6
4. Requerimientos del sistema para el módulo móvil	7
4.1 Requerimientos funcionales.....	7
4.2 Requerimientos no funcionales.....	7
5. Android Studio y su SDK	7
6. React Native y Expo: las herramientas que uso para construir la app	8
7. Diagrama de componentes e identificación de capas.....	9
8. Diagrama de paquetes	10
9. Diagrama de clases	12
10. Mapa de navegación de la aplicación	14
11. Codificación de los módulos.....	15
11.1 Módulo de Autenticación (Login).....	15
11.2 Módulo de Inicio (listado de ofertas).....	16
11.3 Módulo de Trueques	17
11.4 Módulo de Perfil	17
12. Librerías necesarias para la capa de presentación.....	18
13. Componentes reutilizables y patrones de diseño	18
13.1 Patrones de diseño aplicados.....	19
14. Buenas prácticas de codificación aplicadas	19
15. División del código fuente en paquetes	20
16. Pruebas unitarias	21
17. Configuración de servidores y acceso a datos.....	21
18. Ambientes de desarrollo y pruebas	22

19. Control de versiones	22
20. Conclusión	23
21. Referencias.....	24

1. Introducción

Mi nombre es Gerson Jonnathan López Oviedo, ficha 3186647, del programa Tecnólogo en Análisis y Desarrollo de Software. Este documento presenta la evidencia GA8-220501096-AA2-EV02, que corresponde al desarrollo de los módulos de la aplicación móvil de mi proyecto Trueque Match, siguiendo los requerimientos que definí desde el inicio del proyecto.

Esta evidencia es de carácter investigativo. Es decir, en vez de mostrar únicamente capturas de pantalla del código corriendo, aquí explico con mis propias palabras y con ejemplos sencillos los conceptos que tuve que estudiar y aplicar para poder construir los módulos móviles: cómo funciona el entorno de Android, qué herramientas elegí y por qué, cómo organicé el código en capas y en paquetes, qué diagramas guiaron mis decisiones, y qué buenas prácticas apliqué al escribirlo.

Trueque Match es una plataforma que permite a los colombianos intercambiar productos, servicios, conocimientos y experiencias sin necesidad de dinero. Ya tengo el backend funcionando en PHP y la parte web funcionando en el navegador; en esta evidencia me enfoco en la otra puerta de entrada a la misma plataforma: la aplicación para el celular, construida en React Native con Expo y probada en un dispositivo Android real.

2. Análisis

Antes de meterme a programar, me tomé un momento para desglosar la guía de la evidencia, porque tiene muchos puntos y algunos suenan más complicados de lo que realmente son. Aquí explico cada punto con un ejemplo cotidiano, para tenerlo claro antes de avanzar.

Un ejemplo que me sirvió para entender todo el conjunto: armar los módulos móviles de una app es como amoblar un apartamento nuevo. Primero se debe conocer el plano del apartamento (los diagramas de clases, paquetes y componentes), después decidir en qué cuarto va cada mueble (identificar las capas), elegir con qué herramientas voy a armar los muebles (Android Studio, su SDK, las librerías), y por último ensamblar cada mueble con cuidado, revisando que no le falte un tornillo (las buenas prácticas y las pruebas unitarias). Si me salto un paso, el apartamento queda armado, pero coleccionado.

Punto de la guía	Qué significa en la práctica
Estudiar los conceptos del componente formativo	Repasar teoría de desarrollo móvil antes de escribir una sola línea de código.
Conocer los requerimientos del sistema	Tener claro qué debe poder hacer un usuario desde el celular.
Conocer Android Studio y su SDK	Entender la herramienta oficial de Google para compilar apps de Android, aunque yo programe con Expo por encima de ella.
Diagramas de clases, paquetes y componentes	Los planos que muestran qué piezas tiene la app, cómo se agrupan y cómo se conectan.
Identificar las capas de los componentes	Saber si algo pertenece a lo que el usuario ve, a la lógica interna, o a los datos.
Metodología de desarrollo	El orden de trabajo que sigo (Scrum), igual que en el resto del proyecto.
Mapa de navegación	El plano de qué pantalla lleva a cuál cuando el usuario toca un botón.
Codificar cada módulo	Escribir el código real de cada pantalla y función.
Control de versiones	Guardar el historial de cambios con Git, igual que en el resto del proyecto.
Librerías para la capa de presentación	Las piezas ya hechas por otros programadores que uso en vez de reinventar la rueda.
Dividir en componentes reutilizables	No repetir el mismo botón cien veces; hacerlo una vez y usarlo cien veces.
Buenas prácticas de codificación	Escribir código ordenado, comentado y fácil de leer para otra persona (o para mí mismo en seis meses).

Punto de la guía	Qué significa en la práctica
Dividir el código en paquetes	Guardar cada tipo de archivo en su propia carpeta con nombre claro.
Patrones de diseño por componente	Formas ya probadas de organizar el código que evitan errores comunes.
Pruebas unitarias	Comprobar que cada pieza pequeña del código funciona sola, antes de armar el rompecabezas completo.
Configuración de servidores y acceso a datos	Cómo el celular le habla al servidor donde vive la base de datos.
Documentar los ambientes de desarrollo y pruebas	Dejar por escrito en qué computador, sistema operativo y dispositivo se hizo y probó todo.

3. Metodología de desarrollo: Scrum aplicado al módulo móvil

Para organizar el trabajo de este proyecto sigo usando la metodología ágil Scrum, la misma que ya venía aplicando en la parte web. Scrum es un marco de trabajo para desarrollar productos complejos de forma incremental, dividiendo el trabajo en ciclos cortos llamados sprints, en los que un equipo entrega poco a poco partes útiles del producto (Schwaber & Sutherland, 2020).

Un ejemplo para entenderlo: Scrum es como cocinar un almuerzo grande para una fiesta en vez de intentar servirlo todo de un solo golpe al final. Primero sirvo la sopa, la pruebo, ajusto la sal; después el plato fuerte, lo pruebo; después el postre. Si algo sabe mal, lo corrijo en ese momento y no cuando ya until todos los invitados están sentados en la mesa. Cada 'plato' que entrego es un sprint.

Para el módulo móvil, dividí el trabajo en sprints cortos así:

- Sprint 1: configurar el entorno (Node.js, Expo, dispositivo físico conectado) y armar la navegación básica entre pantallas.
- Sprint 2: construir la pantalla de autenticación (Login) y conectarla con la API que ya existe en el backend.
- Sprint 3: construir la pantalla de inicio (listado de ofertas) y el detalle de una oferta.
- Sprint 4: construir el módulo de trueques y el de perfil, además de los componentes reutilizables (botones, tarjetas).
- Sprint 5: pruebas unitarias, revisión de buenas prácticas y documentación final.

En cada sprint yo cumplo los tres roles que describe Scrum al ser un proyecto formativo individual: decido qué se construye (Product Owner), superviso que el proceso se cumpla (Scrum Master) y escribo el código (Developer). Esto no es lo ideal en una empresa real con varias personas, pero para un proyecto de formación me permite igual aplicar la lógica de trabajo por sprints e ir entregando algo funcional cada semana.

4. Requerimientos del sistema para el módulo móvil

Los requerimientos del módulo móvil no son nuevos: son los mismos requerimientos generales de Trueque Match, pero vistos desde la pantalla de un celular en vez de un navegador. Los separo en funcionales (lo que la app debe hacer) y no funcionales (cómo debe comportarse mientras lo hace).

4.1 Requerimientos funcionales

Código	Requerimiento
RF-M01	El usuario debe poder iniciar sesión desde el celular con el mismo correo y contraseña que usa en la web.
RF-M02	El usuario debe poder ver el listado de ofertas publicadas por otras personas.
RF-M03	El usuario debe poder ver el detalle completo de una oferta.
RF-M04	El usuario debe poder proponer y dar seguimiento a un trueque.
RF-M05	El usuario debe poder ver y editar su perfil, incluyendo su reputación.
RF-M06	El usuario debe poder recibir notificaciones de la actividad relacionada con sus ofertas.

4.2 Requerimientos no funcionales

Código	Requerimiento
RNF-M01	La app debe funcionar sobre Android, que es el sistema operativo del dispositivo de pruebas (Redmi 9).
RNF-M02	La app debe consumir las mismas 9 APIs REST del backend PHP, sin duplicar lógica de negocio en el celular.
RNF-M03	La navegación entre pantallas debe sentirse fluida, sin recargas completas de pantalla.
RNF-M04	El código debe quedar organizado en carpetas y componentes reutilizables, no en un solo archivo gigante.

5. Android Studio y su SDK

Android Studio es el entorno de desarrollo (IDE) oficial de Google para construir aplicaciones Android. Viene integrado con el SDK de Android, que es el conjunto de herramientas, bibliotecas y emuladores necesarios para compilar, ejecutar y depurar una app antes de instalarla en un celular real (Google, s.f.-a).

Un ejemplo para entenderlo: si Android es el país donde va a vivir mi app, el SDK es el conjunto de documentos, permisos y traductores que necesito para que mi app pueda entrar y moverse en ese país sin problemas. Android Studio, por su parte, es la oficina completa donde tramito todos esos documentos: ahí escribo el código, reviso errores, y tengo un celular de mentira (el emulador) para probar todo antes de subirme al avión de verdad.

En mi caso, no escribo el código directamente en Android Studio con Java o Kotlin, sino en React Native con Expo, un framework que me permite programar en JavaScript. Sin embargo, esto no significa que el SDK de Android desaparezca: sigue trabajando por debajo. Cuando genero el instalable final de la app (el APK), ese proceso usa piezas del SDK de Android para empaquetar mi código JavaScript dentro de una aplicación que el sistema operativo Android reconoce y puede instalar. Además, cuando uso Expo Go para probar la app en mi Redmi 9 mientras la desarrollo, Android sigue siendo el sistema operativo que recibe y ejecuta esa app de pruebas.

Conocer el SDK también me sirvió para entender conceptos que uso indirectamente a través de Expo, como el nivel de API de Android (la versión mínima de Android que soporta mi app), los permisos que la app debe solicitar (por ejemplo, para notificaciones), y el proceso de firma digital que todo APK necesita antes de poder instalarse fuera de una tienda de aplicaciones.

6. React Native y Expo: las herramientas que uso para construir la app

React Native es un framework de código abierto, creado originalmente por Meta, que permite construir aplicaciones móviles reales usando JavaScript y los mismos conceptos del framework React, en vez de tener que aprender Java o Kotlin para Android y otro lenguaje distinto para iOS (React Native, s.f.). Encima de React Native uso Expo, un conjunto de herramientas que simplifica todavía más el trabajo: me da un servidor de desarrollo, una app llamada Expo Go para probar en el celular sin cables ni configuraciones complicadas, y una librería de módulos ya listos para usar (Expo, s.f.).

Un ejemplo para entenderlo: React Native es como escribir la receta de una comida una sola vez, y que esa misma receta la puedan cocinar tanto en una estufa de gas (Android) como en una eléctrica (iOS), porque el sabor final es igual de real en ambos casos, no una versión light. Expo, en ese mismo

ejemplo, es el kit de cocina que me regalan con todos los utensilios ya listos, para no tener que fabricar yo mismo cada cuchara y cada olla antes de poder empezar a cocinar.

Elegí React Native con Expo por tres razones prácticas: primero, ya sabía JavaScript por el trabajo que hice en el backend y el frontend web del proyecto, así que no tuve que aprender un lenguaje nuevo desde cero. Segundo, Expo me permite ver los cambios en tiempo real en mi Redmi 9 sin necesidad de compilar un APK cada vez que modifico una línea de código. Y tercero, la app móvil puede consumir exactamente las mismas 9 APIs REST del backend PHP, sin escribir lógica de negocio duplicada.

7. Diagrama de componentes e identificación de capas

El diagrama de componentes muestra, a grandes rasgos, las piezas grandes de todo el sistema Trueque Match y cómo se comunican entre sí. Lo incluyo de nuevo aquí porque es clave para entender en qué punto exacto se conecta el módulo móvil con el resto del proyecto: la app Android no habla directamente con la base de datos, siempre pasa por las APIs REST del servidor.

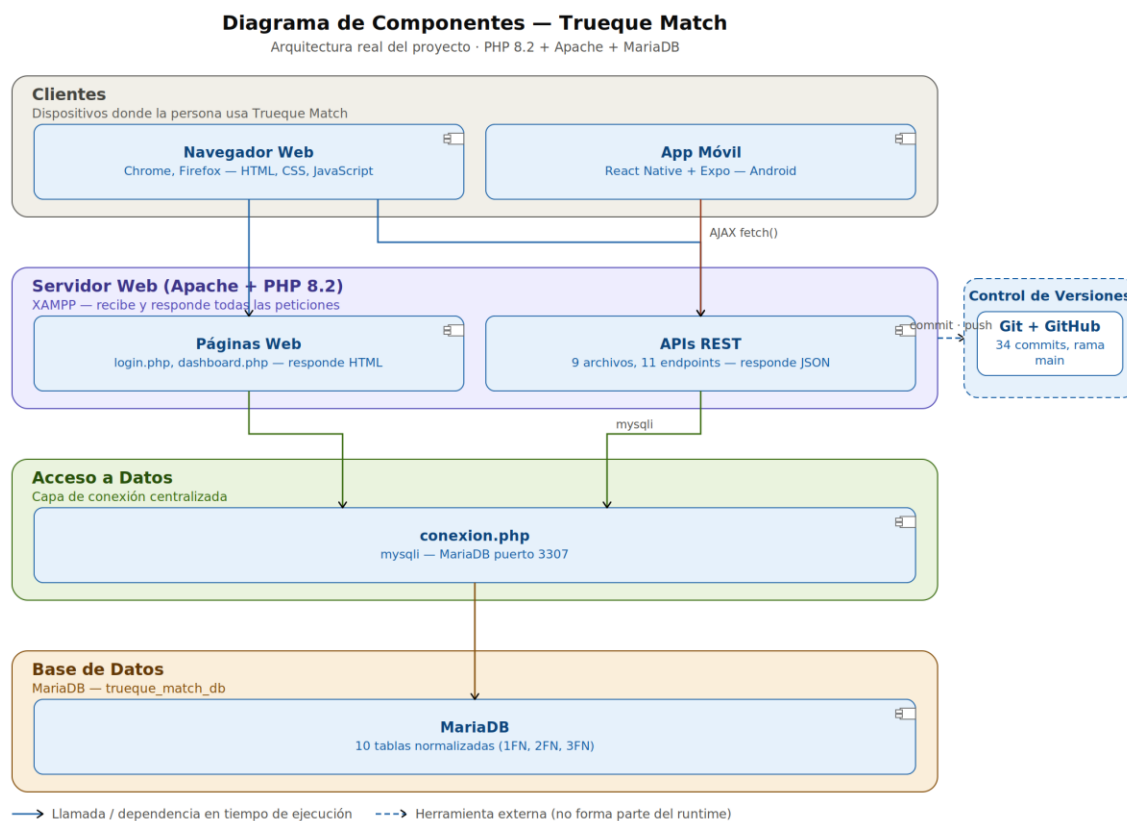


Figura 1. Diagrama de componentes de Trueque Match, con la app móvil como uno de los dos clientes que consumen las APIs REST.

Un ejemplo para entenderlo: pensemos en un restaurante. El navegador web y la app móvil son como dos ventanillas distintas de pedidos (el mesero de mesa y el domiciliario), pero ambos gritan el pedido a la misma cocina (las APIs REST), y esa cocina es la única que puede sacar los ingredientes de la despensa (la base de datos). Ninguna ventanilla entra directo a la despensa a servirse sola.

Con este diagrama identifico claramente en qué capa vive cada componente de la app móvil:

Capa	Dónde vive en la app móvil
Presentación	Las pantallas de React Native (Login, Home, Trueques, Chat, Perfil) y los componentes visuales (botones, tarjetas).
Lógica de negocio	No vive en el celular: vive en las APIs REST de PHP en el servidor. El celular solo pide y muestra resultados.
Acceso a datos	Tampoco vive en el celular: vive en <code>conexion.php</code> , en el servidor, usando <code>mysqli</code> .
Datos	La base de datos MariaDB, en el mismo servidor, nunca directamente accesible desde el celular.

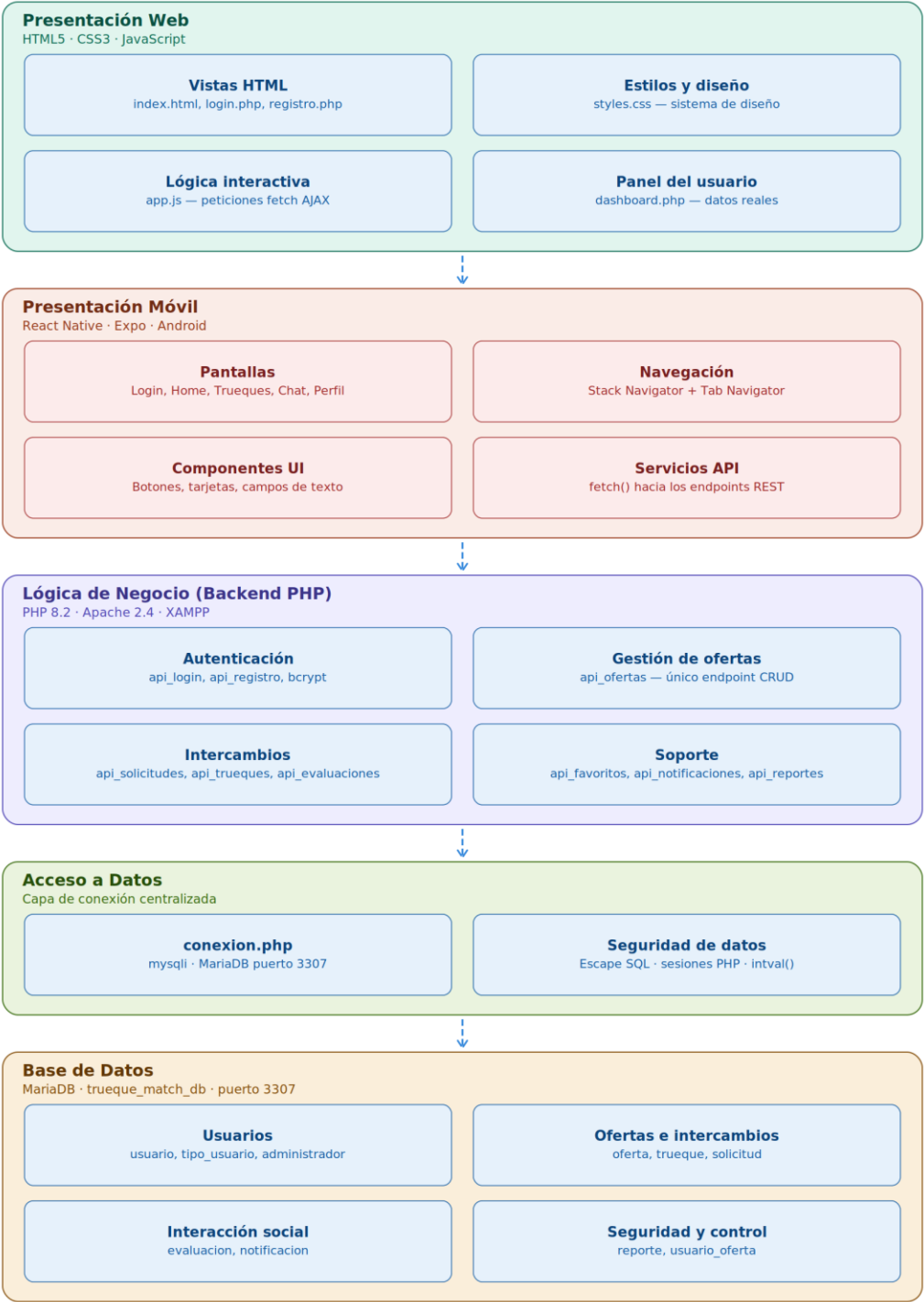
Esta separación es intencional y es una de las decisiones de arquitectura más importantes del módulo móvil: la app Android es 'tonta' a propósito, en el buen sentido. No guarda reglas de negocio ni toca la base de datos directamente. Si mañana cambio de motor de base de datos o reescribo el backend en otro lenguaje, la app móvil no debería enterarse, porque solo conoce las APIs REST, no lo que hay detrás de ellas.

8. Diagrama de paquetes

El diagrama de paquetes organiza todo el código del proyecto, web y móvil, en cinco grandes grupos, ordenados desde lo que el usuario ve hasta donde se guardan los datos. Lo pienso como los cajones de un mueble organizador: cada cajón tiene una etiqueta clara y solo guardo ahí lo que corresponde, para no terminar buscando las medias entre las camisetitas.

Diagrama de Paquetes — Trueque Match

Organización del código en 5 paquetes, de la interfaz hasta la base de datos



□ Subpaquete / componente --> Dependencia («uses»)

Figura 2. Diagrama de paquetes de Trueque Match. El paquete 'Presentación Móvil' agrupa las pantallas, la navegación, los componentes UI y los servicios API de la app Android.

El paquete que más me interesa para esta evidencia es Presentación Móvil, que a su vez se divide en cuatro subgrupos:

- Pantallas: Login, Home, Trueques, Chat y Perfil. Cada una es un archivo independiente.
- Navegación: la configuración del Stack Navigator (para ir de una pantalla a otra 'encima' de la anterior) y del Tab Navigator (para moverse entre las secciones principales con una barra inferior).
- Componentes UI: piezas visuales reutilizables como botones, tarjetas de oferta y campos de texto.
- Servicios API: funciones que usan `fetch()` para hablar con los endpoints REST del backend, separadas de las pantallas para no repetir código de conexión en cada una.

Este paquete depende del paquete de Lógica de Negocio (Backend PHP) exactamente igual que el paquete de Presentación Web: ambos le piden información a las mismas APIs REST y ninguno de los dos toca directamente la Base de Datos ni la capa de Acceso a Datos.

9. Diagrama de clases

El diagrama de clases describe qué información maneja cada entidad del sistema y cómo se relacionan entre sí. En Trueque Match, ese diagrama ya lo construí durante el diseño de la base de datos como parte del Modelo Entidad-Relación (EER), porque en un proyecto de este tamaño las clases principales del backend y del móvil corresponden, casi en espejo, a las tablas ya normalizadas de la base de datos.

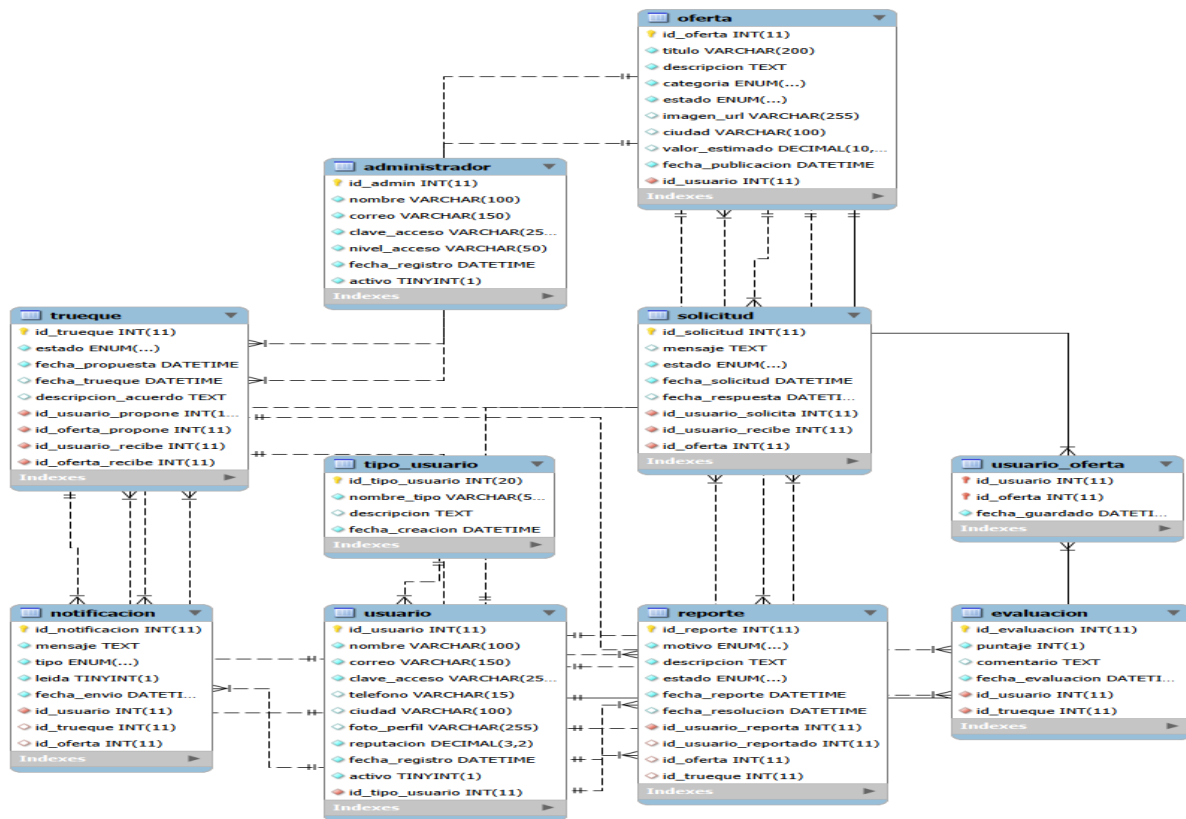


Figura 3. Modelo Entidad-Relación (EER) de trueque_match_db, base para las clases de datos que consume la app móvil.

Un ejemplo para entenderlo: el diagrama de clases y el diagrama entidad-relación son como el molde de una galleta y la galleta ya horneada: el molde (la clase) define la forma, y cada galleta horneada (un objeto o una fila de la base de datos) tiene esa misma forma pero con su propio relleno. Por eso, en la app móvil, cuando pido una oferta a la API y la recibo en JSON, esa respuesta tiene 'la forma' de la clase Oferta, aunque nunca escribí una clase Java o Kotlin llamada Oferta: en JavaScript, esa forma la describo como un objeto plano.

A partir del modelo de datos, en el código móvil trabajé con los siguientes 'moldes' de información, aunque en JavaScript no se declaren como clases formales sino como la forma esperada de cada objeto:

Clase / entidad	Atributos principales	Dónde se usa en el módulo móvil
Usuario	id, nombre, correo, ciudad, reputación	Pantalla de Login y Perfil
Oferta	id, título, descripción, categoría, estado	Pantalla Home y Detalle de Oferta
Trueque	id, oferta_id, estado, fecha	Pantalla Trueques
Solicitud	id, oferta_id, usuario_id, estado	Pantalla Trueques (previo a formalizar)

Clase / entidad	Atributos principales	Dónde se usa en el módulo móvil
Evaluación	id, trueque_id, calificación, comentario	Pantalla Perfil (historial de reputación)
Notificación	id, usuario_id, mensaje, leída	Pantalla Home (badge de notificaciones)

Cada uno de estos 'moldes' llega a la app móvil como la respuesta JSON de una API REST, y ahí se guarda temporalmente en el estado de React (useState) de la pantalla correspondiente, no en una base de datos local del celular.

10. Mapa de navegación de la aplicación

El mapa de navegación es el plano de qué pantalla puede llevar a cuál otra cuando el usuario toca algo en la app. Lo armé usando dos tipos de navegación de la librería React Navigation, que es el estándar de facto para moverse entre pantallas en cualquier app de React Native.

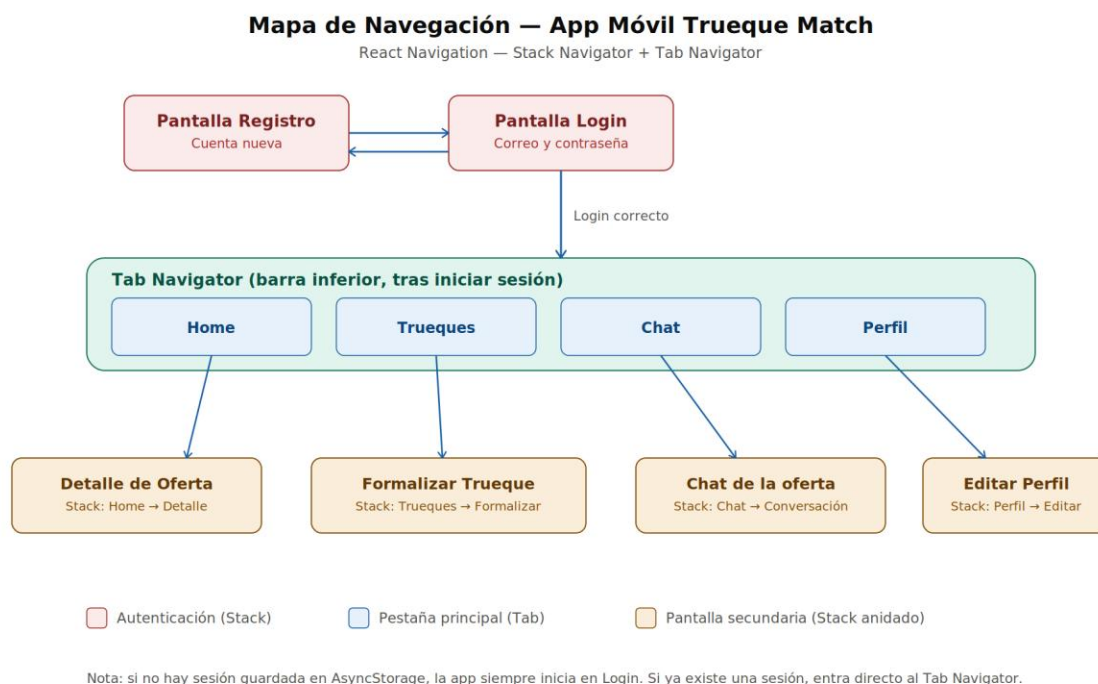


Figura 4. Mapa de navegación de la app móvil Trueque Match, con Stack Navigator para autenticación y pantallas secundarias, y Tab Navigator para las cuatro secciones principales.

Un ejemplo para entenderlo: el Tab Navigator es como los botones de canal de un televisor viejo: siempre están ahí abajo, visibles, y salto de un canal a otro sin perder el hilo de dónde estoy. El Stack Navigator, en cambio, es como una pila de hojas sobre un escritorio: cuando abro el detalle de una oferta,

pongo una hoja nueva encima de la pila, y cuando toco 'volver', simplemente quito esa hoja de encima y aparece la que estaba debajo.

En la práctica, cuando la app arranca reviso si ya existe una sesión guardada; si no existe, el usuario ve la pantalla de Login o Registro (navegación tipo Stack). Una vez inicia sesión correctamente, entra al Tab Navigator con las cuatro secciones principales (Home, Trueques, Chat, Perfil), y desde cada una de esas pestañas puede abrir pantallas secundarias que se apilan encima, como el detalle de una oferta o la conversación de un chat.

11. Codificación de los módulos

A continuación explico cómo codifiqué cada módulo de la app móvil. En cada uno incluyo un fragmento de código representativo, comentado línea por línea, que muestra la estructura real que uso, no un pseudocódigo genérico.

11.1 Módulo de Autenticación (Login)

Este módulo pide correo y contraseña, y llama a la misma API de login que ya usa la versión web (api_login.php), reutilizando toda la lógica de validación que vive en el servidor.

```
// screens/LoginScreen.js
import { useState } from 'react';
import { View, TextInput, Button, Text } from 'react-native';
import { loginUsuario } from '../services/authService';

export default function LoginScreen({ navigation }) {
  // Guardo lo que el usuario escribe en el correo y la contraseña
  const [correo, setCorreo] = useState("");
  const [clave, setClave] = useState("");
  const [error, setError] = useState("");

  async function manejarLogin() {
    // Le pido al servicio que hable con api_login.php por mí
    const resultado = await loginUsuario(correo, clave);
    if (resultado.ok) {
      navigation.replace('TabPrincipal'); // entro a la app
    } else {
      setError('Correo o contraseña incorrectos');
    }
  }
}
```

```

}

return (
  <View>
    <TextInput placeholder='Correo' onChangeText={setCorreo} />
    <TextInput placeholder='Contraseña' secureTextEntry onChangeText={setClave} />
    {error ? <Text>{error}</Text> : null}
    <Button title='Ingresar' onPress={manejarLogin} />
  </View>
);
}

```

11.2 Módulo de Inicio (listado de ofertas)

La pantalla Home pide el listado de ofertas a la API única de ofertas (`api_ofertas.php`) apenas se monta el componente, y muestra cada oferta dentro de una tarjeta reutilizable.

```

// screens/HomeScreen.js
import { useEffect, useState } from 'react';
import { FlatList } from 'react-native';
import TarjetaOferta from '../components/TarjetaOferta';
import { obtenerOfertas } from '../services/ofertaService';

export default function HomeScreen({ navigation }) {
  const [ofertas, setOfertas] = useState([]);

  useEffect(() => {
    // Apenas aparece la pantalla, pido las ofertas al servidor
    obtenerOfertas().then(setOfertas);
  }, []);

  return (
    <FlatList
      data={ofertas}
      keyExtractor={(item) => String(item.id)}
      renderItem={({ item }) => (
        <TarjetaOferta
          oferta={item}
          onPress={() => navigation.navigate('DetalleOferta', { id: item.id })}
        />
      )}
    />
  )
}

```

```
    />  
  );  
}
```

11.3 Módulo de Trueques

Este módulo lista las solicitudes y trueques del usuario, y permite avanzar el estado de un trueque (proponer, aceptar, completar), consumiendo `api_solicitudes.php` y `api_trueques.php`.

```
// services/truequeService.js  
import { API_BASE_URL } from '../config';  
  
// Todas las pantallas de trueques usan esta misma función,  
// así evito repetir la URL base y el manejo de errores en cada pantalla.  
export async function completarTrueque(truequeId) {  
  const respuesta = await fetch(`${API_BASE_URL}/api_trueques.php`, {  
    method: 'PUT',  
    headers: { 'Content-Type': 'application/json' },  
    body: JSON.stringify({ id: truequeId, accion: 'completar' }),  
  });  
  return respuesta.json();  
}
```

11.4 Módulo de Perfil

La pantalla de Perfil muestra los datos del usuario y su reputación, calculada automáticamente en el backend a partir de las evaluaciones, la misma lógica que ya construí para la versión web.

```
// screens/PerfilScreen.js  
import { useEffect, useState } from 'react';  
import { View, Text } from 'react-native';  
import { obtenerPerfil } from '../services/usuarioService';  
  
export default function PerfilScreen() {  
  const [usuario, setUsuario] = useState(null);  
  
  useEffect(() => {  
    obtenerPerfil().then(setUsuario);  
  }, []);  
  
  if (!usuario) return null; // mientras carga, no muestro nada roto
```

```

return (
  <View>
    <Text>{usuario.nombre}</Text>
    <Text>Reputación: {usuario.reputacion} / 5</Text>
  </View>
);
}

```

12. Librerías necesarias para la capa de presentación

Para la capa de presentación de la app móvil no reinvento la rueda: uso librerías ya probadas por la comunidad de React Native, cada una resolviendo un problema puntual.

Librería	Para qué la uso
React Navigation	Stack Navigator y Tab Navigator, el corazón del mapa de navegación.
Expo (SDK)	Servidor de desarrollo, Expo Go para probar en el Redmi 9 sin cables, y módulos nativos listos para usar.
React Native (core)	Componentes básicos de interfaz: View, Text, TextInput, FlatList, Button.
AsyncStorage	Guardar la sesión del usuario en el celular, para no pedirle el login cada vez que abre la app.
Fetch API (nativa)	Hacer las peticiones HTTP hacia las 9 APIs REST del backend, sin necesidad de una librería extra.

13. Componentes reutilizables y patrones de diseño

Dividí la interfaz en componentes pequeños y reutilizables en vez de escribir cada pantalla como un bloque único de código. Esto es exactamente lo mismo que ya hice en el frontend web con las tarjetas de oferta, solo que ahora aplicado a React Native.

Un ejemplo para entenderlo: es como los bloques de LEGO. En vez de esculpir una casa completa de una sola pieza, armo piezas pequeñas y estandarizadas (un botón, una tarjeta, un campo de texto) y las combino en distintas pantallas. Si mañana quiero que todos los botones de la app cambien de color, cambio la pieza LEGO una sola vez, no las cien casas que ya armé con ella.

```

// components/TarjetaOferta.js
// Este componente se reutiliza en Home, en Favoritos y en Búsqueda:
// tres pantallas distintas, un solo archivo que mantener.
import { View, Text, TouchableOpacity } from 'react-native';

```

```
export default function TarjetaOferta({ oferta, onPress }) {
  return (
    <TouchableOpacity onPress={onPress}>
      <View>
        <Text>{oferta.titulo}</Text>
        <Text>{oferta.categoria}</Text>
      </View>
    </TouchableOpacity>
  );
}
```

13.1 Patrones de diseño aplicados

- Patrón de componentes (Component Pattern): cada pieza visual es independiente, recibe datos por props y no sabe nada del resto de la app; así puedo probarla y reutilizarla por separado.
- Separación contenedor / presentación: las pantallas (screens) se encargan de pedir datos y manejar el estado; los componentes (components) solo se encargan de mostrar lo que reciben, sin pedir nada por su cuenta.
- Capa de servicios (Service Layer): ninguna pantalla llama a fetch() directamente. Todas pasan por un archivo de servicio (authService, ofertaService, truequeService), igual que en el backend toda consulta pasa por conexion.php. Esto evita repetir la URL base y el manejo de errores en cada pantalla.

14. Buenas prácticas de codificación aplicadas

Además de que el código funcione, me propuse que fuera fácil de leer y de mantener, siguiendo prácticas que ya venía aplicando en el backend PHP y que trasladé al mundo de React Native.

- Nombres descriptivos: una función se llama obtenerOfertas() y no fl() o getData(), para que cualquiera entienda qué hace sin tener que leer todo su contenido.
- Comentarios línea por línea en las partes que no son obvias a simple vista, explicando el porqué de una decisión y no solo el qué.
- Una responsabilidad por archivo: una pantalla no hace peticiones HTTP directamente, un componente no decide a dónde navegar, un servicio no dibuja interfaz.

- Manejo explícito de los estados de carga y error, en vez de asumir que la petición al servidor siempre va a responder rápido y sin fallas.
- Reutilización antes que copiar y pegar: si necesito el mismo botón en dos pantallas, lo saco a un componente en vez de duplicar el código.
- Indentación y formato consistente en todo el proyecto, para que el código se lea igual sin importar en qué archivo esté.

15. División del código fuente en paquetes

Organicé las carpetas del proyecto móvil con nombres que explican su contenido con solo leerlos, siguiendo la misma idea del diagrama de paquetes mostrado en la sección 8.

```

TruequeMatchApp/
├── App.js           // Punto de entrada, arma la navegación
├── config.js        // URL base de la API y constantes
├── navigation/
│   ├── StackNavigator.js // Login, Registro, pantallas secundarias
│   └── TabNavigator.js   // Home, Trueques, Chat, Perfil
├── screens/
│   ├── LoginScreen.js
│   ├── HomeScreen.js
│   ├── TruequesScreen.js
│   └── PerfilScreen.js
├── components/
│   ├── TarjetaOferta.js
│   ├── BotonPrimario.js
│   └── CampoTexto.js
├── services/
│   ├── authService.js
│   ├── ofertaService.js
│   ├── truequeService.js
│   └── usuarioService.js
├── __tests__/
│   └── ofertaService.test.js

```

Con esta organización, si necesito arreglar un error en cómo se piden las ofertas, sé que solo debo abrir `services/ofertaService.js`, sin tener que revisar pantalla por pantalla buscando dónde quedó ese código.

16. Pruebas unitarias

Para las pruebas unitarias de la app móvil uso Jest, un framework de pruebas para JavaScript que ya viene integrado por defecto en los proyectos de React Native y que permite comprobar que una función pequeña hace exactamente lo que se espera de ella, de forma aislada (Jest, s.f.).

Un ejemplo para entenderlo: una prueba unitaria es como probar cada ingrediente por separado antes de meterlo a la olla: pruebo que la sal sabe a sal y el azúcar sabe a azúcar, antes de mezclarlos y descubrir demasiado tarde que me confundí de frasco.

Escribí pruebas para las funciones de los servicios, que son las que más se prestan para fallar de forma silenciosa si cambia algo en el backend. Por ejemplo, esta prueba comprueba que la función que arma la URL de una oferta específica la construye correctamente:

```
// __tests__/ofertaService.test.js
import { construirUrlOferta } from '../services/ofertaService';

test('arma la URL de una oferta con su id', () => {
  const url = construirUrlOferta(21);
  expect(url).toBe('http://192.168.1.10/.../api_ofertas.php?id=21');
});
```

Este tipo de prueba es rápida de ejecutar y me avisa de inmediato si, por accidente, cambio el nombre de un parámetro o rompo la construcción de una URL, sin tener que abrir la app completa en el celular cada vez para descubrirlo.

17. Configuración de servidores y acceso a datos

La app móvil no tiene servidor propio: consume el mismo backend PHP que ya usa la versión web, corriendo sobre XAMPP en mi computador, con MariaDB en el puerto 3307 en vez del puerto por defecto 3306, porque ese puerto ya estaba en uso por otro programa.

Un detalle importante que aprendí al conectar el celular: mientras que en el navegador del mismo computador puedo usar 'localhost' para llegar al servidor, el celular es un dispositivo distinto conectado por wifi, así que 'localhost' ahí significaría 'el propio celular', no mi computador. Por eso, en config.js de la app móvil, la URL base de la API apunta a la dirección IP local de mi computador dentro de la red wifi de la casa (por ejemplo, 192.168.1.10), y no a 'localhost'.

```
// config.js
// OJO: en el celular NO sirve 'localhost', porque ahí 'localhost'
// sería el propio celular, no el computador con XAMPP.
export const API_BASE_URL = 'http://192.168.1.10/proyecto_sena_nuevo/app_trueque_match';
```

Esto significa que, para que la app móvil funcione en modo de pruebas, el celular y el computador deben estar conectados a la misma red wifi, y el firewall de Windows debe permitir conexiones entrantes al puerto de Apache. Documento esta dependencia aquí para no olvidarla si algún día cambio de red.

18. Ambientes de desarrollo y pruebas

Elemento	Configuración
Sistema operativo de desarrollo	Windows (Laptop Acer Core 7)
Entorno de la app móvil	React Native + Expo
Herramienta de pruebas en el celular	Expo Go, sobre Android
Dispositivo físico de pruebas	Redmi 9 (Android)
Servidor consumido por la app	Apache 2.4 + PHP 8.2 (XAMPP), puerto 80
Motor de base de datos	MariaDB, puerto 3307 (personalizado)
Ruta del proyecto móvil	C:\Users\Sandra\Desktop\TruequeMatchApp
Framework de pruebas unitarias	Jest
Repositorio remoto	GitHub - github.com/lopezoviedogersonjonnathan-rgb/proyecto_sena

El ambiente de desarrollo y el ambiente de pruebas coinciden en la misma red doméstica, lo cual es habitual en proyectos formativos de este tamaño. Todavía no existe un ambiente de producción público para la app móvil, ya que el proyecto está en etapa de aprendizaje dentro del programa de formación SENA.

19. Control de versiones

El código de la app móvil se maneja con el mismo flujo de Git que uso en el resto del proyecto: git add, git commit y git push, en ese orden, siempre desde la carpeta correspondiente. Cada commit describe con precisión qué módulo se agregó o se corrigió, siguiendo la misma convención de prefijos que uso en el backend: feat para una funcionalidad nueva, fix para una corrección, y docs para cambios de documentación.

Mantener el mismo estándar de commits entre el backend, el frontend web y la app móvil me permite revisar el historial completo del proyecto sin tener que recordar reglas distintas según en qué carpeta esté trabajando.

20. Conclusión

Desarrollar los módulos móviles de Trueque Match me sirvió para confirmar algo que ya venía notando en la parte web del proyecto: separar bien las capas y los paquetes desde el principio hace que agregar una plataforma nueva (en este caso, Android) sea mucho más simple de lo que suena. La app móvil no necesitó reinventar reglas de negocio ni acceso a datos, porque esas piezas ya existían y viven en un solo lugar: las APIs REST del backend.

Cada módulo que codifiqué (autenticación, inicio, trueques y perfil) sigue el mismo patrón: una pantalla que pide datos a un servicio, un servicio que le habla a una API, y componentes reutilizables que solo se preocupan por mostrar lo que reciben. Documenté también las decisiones que no son obvias a simple vista, como el uso de la dirección IP local en vez de localhost, porque son justo el tipo de detalle que se me olvidaría a mí mismo si no quedara escrito.

Quedan como mejoras futuras compilar el APK final firmado para distribución fuera de Expo Go, ampliar la cobertura de pruebas unitarias a los componentes de interfaz (no solo a los servicios), y evaluar el uso de notificaciones push nativas para avisar en tiempo real sobre nuevas solicitudes de trueque.

21. Referencias

Expo. (s.f.). *Expo documentation*. Recuperado el 13 de julio de 2026, de <https://docs.expo.dev/>

Google. (s.f.-a). *Download Android Studio & App Tools*. Android Developers. Recuperado el 13 de julio de 2026, de <https://developer.android.com/studio>

Google. (s.f.-b). *Develop for Android*. Android Developers. Recuperado el 13 de julio de 2026, de <https://developer.android.com/develop>

Jest. (s.f.). *Jest: Delightful JavaScript testing*. Recuperado el 13 de julio de 2026, de <https://jestjs.io/>

Meta Platforms. (s.f.). *React Native documentation*. Recuperado el 13 de julio de 2026, de <https://reactnative.dev/docs/getting-started>

Schwaber, K., & Sutherland, J. (2020). *La guía de Scrum: La guía definitiva de Scrum: Las reglas del juego*. Scrum Guides.

<https://scrumguides.org/>